

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
AI(Artificial Intelligence)
Practical VIII

Roll No.:T006	Name:Nandini Chaudhary
Class:TYIT	Batch:01
Date of Assignment:22-08-2026	Date/Time of Submission:22-08-2026

Aim: Ethics and Bias Analysis

Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class imbalance.

Dataset: Titanic train.csv from Kaggle

1. Load and Explore the Dataset

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("titanic_train.csv")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns)
print("\nDataset Information:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
```

Output:

```
First 5 rows:
   PassengerId  Survived  Pclass
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

   Name                               Sex  Age  SibSp
0  Braund, Mr. Owen Harris             male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0      1
2  Heikkinen, Miss. Laina              female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0      1
4  Allen, Mr. William Henry            male  35.0      0

   Parch  Ticket  Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0   PC 17599  51.0000   C85      C
2      0  STON/O2, 3101282   53.0000   NaN      S
3      0  113803  53.1000  C123      S
4      0  373450   8.0500   NaN      S

Dataset Shape:
(891, 12)

min 0.000000  0.000000  7.250000
max 0.000000  14.454200  51.000000
min 0.000000  0.000000  0.000000
max 0.000000  512.329200
```

2. Check Data Quality

Code:

```
print("Missing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
```

Output:

Missing Values:		Duplicate Rows:	
PassengerId	0	0	
Survived	0		
Pclass	0	Data Types:	
Name	0	PassengerId	int64
Sex	0	Survived	int64
Age	177	Pclass	int64
SibSp	0	Name	str
Parch	0	Sex	str
Ticket	0	Age	float64
Fare	0	...	
Cabin	687	Fare	float64
Embarked	2	Cabin	str
dtype: int64		Embarked	str
		dtype: object	

3. Identify the Target Variable

Code:

```
target = "Survived"
print("Target Variable:", target)
print("\nTarget Values:")
print(df[target].value_counts())
```

Output:

```
Target Variable: Survived

Target Values:
Survived
0      549
1      342
Name: count, dtype: int64
```

4. Analyze Class Distribution

Code:

```
print(df["Survived"].value_counts())
print("\nClass Distribution (%):")
print(df["Survived"].value_counts(normalize=True) * 100)
```

Output:

```
Survived
0      549
1      342
Name: count, dtype: int64

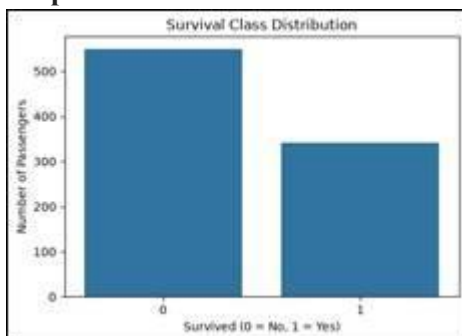
Class Distribution (%):
Survived
0      61.616162
1      38.383838
Name: proportion, dtype: float64
```

5. Visualize Class Imbalance

Code:

```
plt.figure(figsize=(6,4))
sns.countplot(x="Survived", data=df)
plt.title("Survival Class Distribution")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")
plt.show()
```

Output:



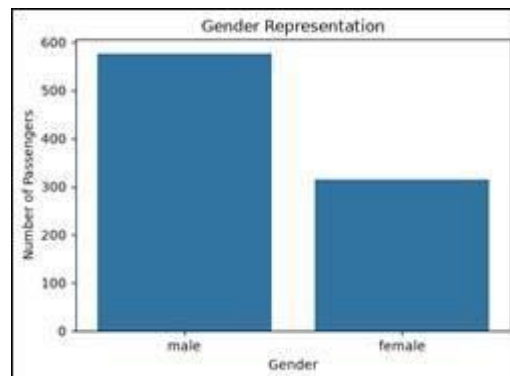
6. Analyze Gender Representation and display graph

Code:

```
print("Gender Representation:")
print(df["Sex"].value_counts())
plt.figure(figsize=(6,4))
sns.countplot(x="Sex", data=df)
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.show()
```

Output:

```
Gender Representation:
Sex
male      577
female    314
Name: count, dtype: int64
```



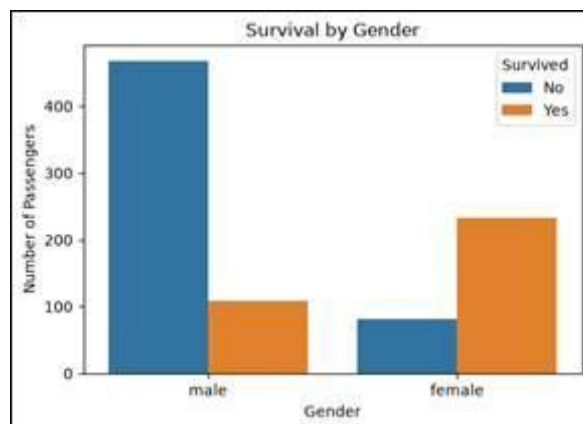
7. Analyze Survival by Gender and display graph

Code:

```
print("Survival by Gender:")
print(pd.crosstab(df["Sex"], df["Survived"]))
plt.figure(figsize=(6,4))
sns.countplot(x="Sex", hue="Survived", data=df)
plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
```

Output:

```
Survival by Gender:
Survived  0    1
Sex
female    81  233
male     468  109
```



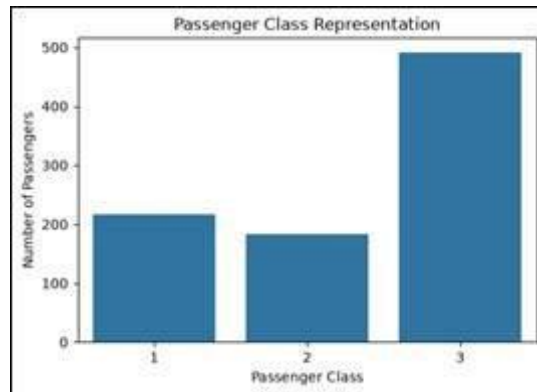
8. Analyze Passenger-Class Representation and display graph

Code:

```
print("Passenger Class Representation:")
print(df["Pclass"].value_counts().sort_index())
plt.figure(figsize=(6,4))
sns.countplot(x="Pclass", data=df)
plt.title("Passenger Class Representation")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.show()
```

Output:

```
Passenger Class Representation:
Pclass
1      216
2      184
3      491
Name: count, dtype: int64
```



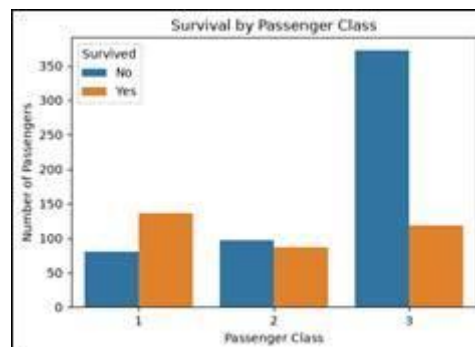
9. Analyze Survival by Passenger Class and display graph

Code:

```
print("\nCredit Risk by Age Group:")
print(pd.crosstab(df["AgeGroup"], df["kredit"]))
plt.figure(figsize=(8,4))
sns.countplot(data=df, x="AgeGroup", hue="kredit")
plt.title("Credit Risk by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.show()
```

Output:

```
Survival by Passenger Class:
Survived  0    1
Pclass
1          80  136
2          97   87
3         372  119
```



10. Analyze Age Groups

Code:

```
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0, 12, 19, 59, 100],
    labels=["Child", "Teenager", "Adult", "Senior"]
)
print("Age Group Distribution:")
print(df["AgeGroup"].value_counts().sort_index())
```

Output:

```
Age Group Distribution:
AgeGroup
Child      69
Teenager   95
Adult     524
Senior     26
Name: count, dtype: int64
```

11. Analyze Survival by Age Group and display graph

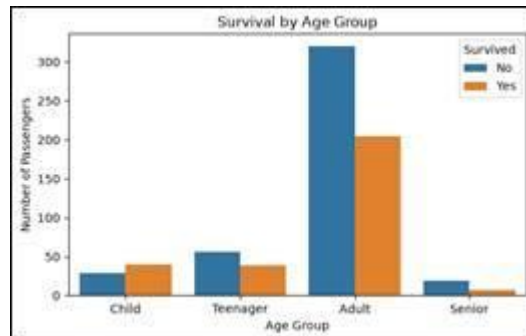
Code:

```
print("Survival by Age Group:")
print(pd.crosstab(df["AgeGroup"], df["Survived"]))
```

```
plt.figure(figsize=(7,4))
sns.countplot(
    x="AgeGroup",
    hue="Survived",
    data=df,
    order=["Child", "Teenager", "Adult", "Senior"]
)
plt.title("Survival by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
```

Output:

Survival by Age Group:		
Survived	0	1
AgeGroup		
Child	29	40
Teenager	56	39
Adult	320	204
Senior	19	7



12. Identify Possible Bias

Ans:

1. Gender bias may be present.
2. Passenger class may affect survival.
3. Age groups have different survival rates.
4. Missing data can also introduce bias.

13. Impact on ML

Ans:

1. Bias in data can make ML predictions less accurate or unfair.
2. The model may learn unwanted patterns from gender, age, or class.

14. Suggested solutions

Ans:

1. Handle missing values properly.
2. Balance the classes if required.
3. Remove duplicate or incorrect data.
4. Check model performance for different groups.

15. Final conclusion

Ans:

The Titanic dataset shows that gender, age, and passenger class affected survival. Proper data cleaning and bias handling are important for building a fair and reliable ML model.

Dataset: German Credit Risk Dataset from Kaggle

1. Load and Explore the Dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_csv("german_credit_data.csv")
# Remove spaces from column names
df.columns = df.columns.str.strip()

# 1. Load and Explore Dataset
print("First 5 Rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
```

```
print(df.columns.tolist())
print("\nDataset Information:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
```

```
First 5 Rows:
   laufkont  laufzeit  moral  verw  hoehe  sparkont  beszeit  rate  famges  \
0         1         18     4     2   1049         1         2     4         2
1         1          9     4     0   2799         1         3     2         3
2         2         12     2     9    841         2         4     2         2
3         1         12     4     0   2122         1         3     3         3
4         1         12     4     0   2171         1         3     4         3

   buerge  ...  verm  alter  weitkred  wohn  bishkred  beruf  pers  telef  \
0         1  ...    2     21         3     1         1     3     2         1
1         1  ...    1     36         3     1         2     3     1         1
2         1  ...    1     23         3     1         1     2     2         1
3         1  ...    1     39         3     1         2     2     1         1
4         1  ...    2     38         1     2         2     2     2         1
```

```
   gastarb  kredit
0         2         1
1         2         1
2         2         1
3         1         1
4         1         1

[5 rows x 21 columns]
```

Dataset Shape:
(1000, 21)

Column Names:
['laufkont', 'laufzeit', 'moral', 'verw', 'hoehe', 'sparkont',

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 21 columns):
#   Column      Non-Null Count  Dtype
---  -
0   laufkont    1000 non-null   int64
1   laufzeit    1000 non-null   int64
2   moral       1000 non-null   int64
3   verw        1000 non-null   int64
4   hoehe       1000 non-null   int64
5   sparkont    1000 non-null   int64
6   beszeit     1000 non-null   int64
7   rate        1000 non-null   int64
8   famges      1000 non-null   int64
...
```

2. Check Data Quality

```
# 2. Check Data Quality
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
# 3. Identify the Target Variable
target = "kredit"
print("\nTarget Variable:", target)
```

```
kredit
1    70.0
0    30.0
```

4. Analyze Class Distribution

```
# 4. Analyze Class Distribution
print("\nCredit Risk Distribution:")
print(df["kredit"].value_counts())
print("\nCredit Risk Percentage:")
print(df["kredit"].value_counts(normalize=True) * 100)
```

```
Name: proportion, dtype: float64
```

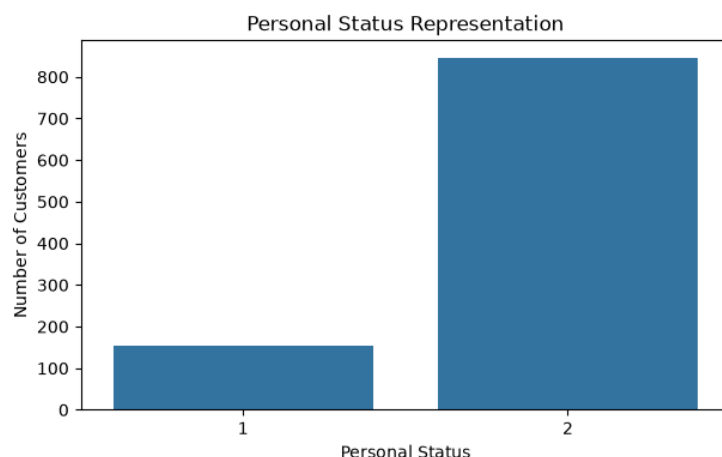
5. Visualize Class Imbalance

```
# 5. Visualize Class Imbalance
plt.figure(figsize=(6,4))
sns.countplot(data=df, x="kredit")
plt.title("Credit Risk Class Distribution")
plt.xlabel("Credit Risk")
plt.ylabel("Number of Customers")
plt.show()
```



6. Analyze Gender Representation and display graph

```
# 6. Analyze Gender / Personal Status Representation
print("\nPersonal Status Representation:")
print(df["pers"].value_counts())
plt.figure(figsize=(7,4))
sns.countplot(data=df, x="pers")
plt.title("Personal Status Representation")
plt.xlabel("Personal Status")
plt.ylabel("Number of Customers")
plt.show()
```



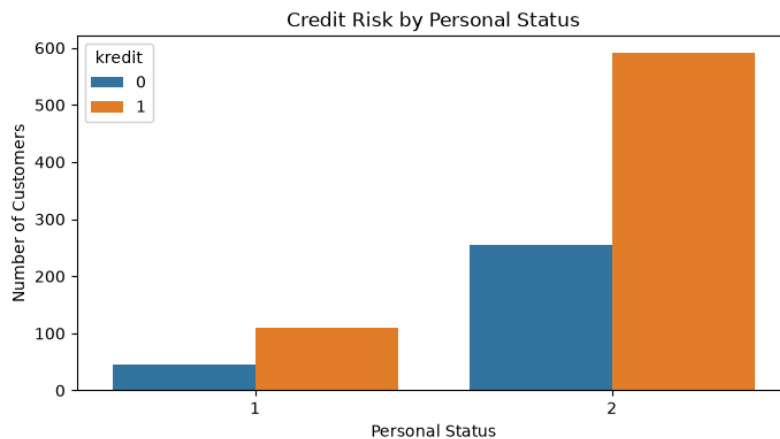
```
Personal Status Representation:
pers
2    845
1    155
Name: count, dtype: int64
```

7. Analyze Credit Risk by Gender and display graph

```
# 7. Analyze Credit Risk by Personal Status
print("\nCredit Risk by Personal Status:")
print(pd.crosstab(df["pers"], df["kredit"]))
plt.figure(figsize=(8,4))
sns.countplot(data=df, x="pers", hue="kredit")
```

```
plt.title("Credit Risk by Personal Status")
plt.xlabel("Personal Status")
plt.ylabel("Number of Customers")
plt.show()
```

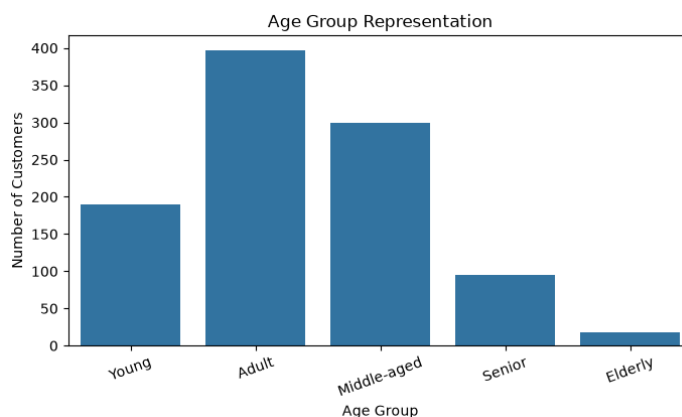
Credit Risk by Personal Status:		
credit	0	1
pers		
1	46	109
2	254	591



8. Analyze Age Groups and display graph

```
# 8. Analyze Age Groups
df["AgeGroup"] = pd.cut(
    df["alter"],
    bins=[0, 25, 35, 50, 65, 100],
    labels=["Young", "Adult", "Middle-aged", "Senior", "Elderly"]
)
print("\nAge Group Representation:")
print(df["AgeGroup"].value_counts().sort_index())
plt.figure(figsize=(8,4))
sns.countplot(data=df, x="AgeGroup")
plt.title("Age Group Representation")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.show()
```

Age Group Representation:	
AgeGroup	
Young	190
Adult	397
Middle-aged	300
Senior	95
Elderly	18
Name: count, dtype: int64	

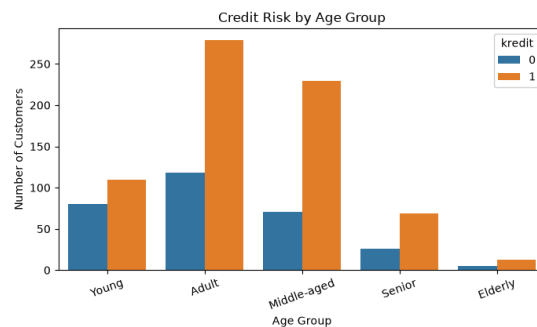


9. Analyze Credit Risk by Age Group and display graph

```
# 9. Analyze Credit Risk by Age Group
print("\nCredit Risk by Age Group:")
print(pd.crosstab(df["AgeGroup"], df["credit"]))
plt.figure(figsize=(8,4))
```

```
sns.countplot(data=df, x="AgeGroup", hue="kredit")
plt.title("Credit Risk by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.show()
```

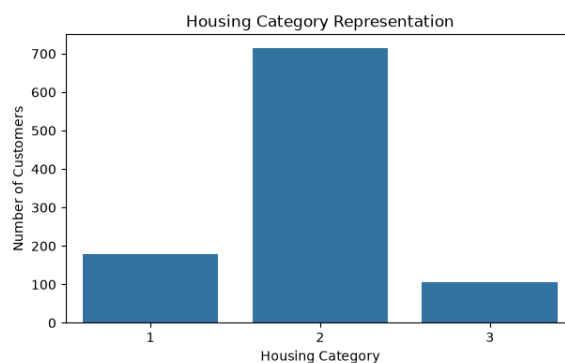
Credit Risk by Age Group:		
kredit	0	1
AgeGroup		
Young	80	110
Adult	118	279
Middle-aged	71	229
Senior	26	69
Elderly	5	13



10. Analyze Housing Category Representation and display graph

```
# 10. Analyze Housing Category Representation
print("\nHousing Category Representation:")
print(df["wohn"].value_counts())
plt.figure(figsize=(7,4))
sns.countplot(data=df, x="wohn")
plt.title("Housing Category Representation")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.show()
```

Housing Category Representation:	
wohn	
2	714
1	179
3	107
Name: count, dtype: int64	



11. Analyze Credit Risk by Housing Category and display graph

```
# 11. Analyze Credit Risk by Housing Category
print("\nCredit Risk by Housing Category:")
print(pd.crosstab(df["wohn"], df["kredit"]))
plt.figure(figsize=(8,4))
sns.countplot(data=df, x="wohn", hue="kredit")
plt.title("Credit Risk by Housing Category")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.show()
```

Credit Risk by Housing Category:		
kredit	0	1
wohn		
1	70	109
2	186	528
3	44	63



12. Identify Possible Bias

- Credit risk classes may be imbalanced.
- Some age, personal-status, or housing groups may be underrepresented.
- This can create unfair patterns in the data.

13. Impact on ML

- The model may favor the majority class.
- Predictions may be less accurate for underrepresented groups.
- Sensitive features may lead to biased decisions.

14. Suggested Solutions

- Balance the classes.
- Handle missing data properly.
- Use stratified sampling.
- Check model performance across different groups.

15. Final Conclusion

The dataset may contain imbalance and group-related bias. Proper preprocessing, balancing, and fairness checks can help build a more reliable ML model.